



KNN DECISION TREE & SVM

RANGKUMAN • 21 SEKSI TRANSKRIP

Tiga cara mesin mengenali pola: dari **kemiripan** tetangga, dari **aturan** pohon keputusan, sampai **margin** terlebar SVM.

01

k-Nearest Neighbor

Mirip tetangga terdekat — hitung jarak, urutkan, voting.

02

Decision Tree

Entropi & information gain — atribut paling informatif menang.

03

Support Vector Machine

Hyperplane bermargin maksimum — plus kernel trick.

AGENDA

TIGA TOPIK, SATU ALUR BELAJAR

Rangkuman inti dari 21 seksi transkrip: definisi, cara kerja, contoh hitung, dan isu praktis saat menerapkan.

01

7 SEKSI

k-Nearest Neighbor

- Lazy vs eager learning
- Intuisi & Voronoi tessellation
- Ukuran jarak: Euclidean s.d. Mahalanobis
- Contoh hitung kertas tisu
- Memilih k, normalisasi, efisiensi

02

8 SEKSI

Decision Tree

- Anatomi pohon: root, branch, leaf
- Studi kasus 14 sampel main tenis
- Entropi & information gain
- Pohon menjadi set of rules
- Overfitting, pruning, C4.5, one-hot

03

6 SEKSI

Support Vector Machine

- Hyperplane, margin, support vector
- Derivasi matematis margin
- Contoh linear: hitung w, b, prediksi
- Kernel trick & kernel polinomial
- Implementasi Python (scikit-learn)

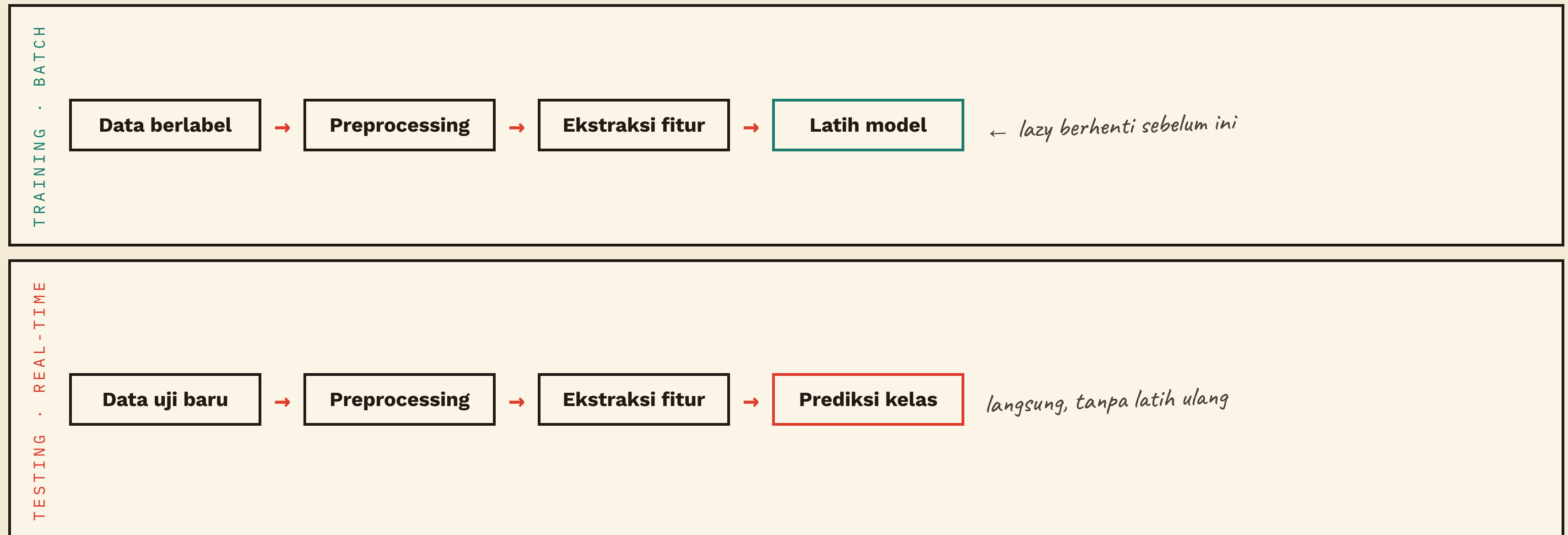
ALUR kemiripan (lazy) → aturan (eager) → margin (eager)

urutannya penting!

KONTEKS

DUA PROSES: LATIH DULU, UJI KEMUDIAN

Semua metode memakai pipeline yang sama — pembedanya: kapan komputasi besar dijalankan.



Batch learning

Proses pembelajaran dijalankan lebih dulu, bisa offline, hasilnya model (eager) atau sekadar feature vector yang disimpan (lazy).

Real-time process

Begitu data uji datang, prediksi dijalankan. Metode eager tinggal memakai model; metode lazy baru mulai menghitung.

KONSEP KUNCI

LAZY VS EAGER LEARNING

Keduanya supervised learning. Bedanya bukan pada hasil akhir, tapi pada kapan biaya komputasi dibayar.

LAZY • KNN

“Nanti saja saat ditanya.”

- Prediksi = menghitung kemiripan ke seluruh sampel latih.
- Batch learning berhenti di ekstraksi fitur — hasilnya disimpan.
- Tanpa prior knowledge soal distribusi data.
- Dikenal juga sebagai **instance-based learning**.

1 data uji × 100 latih = 100 × hitung jarak
10 data uji × 100 latih = 1.000 × hitung jarak

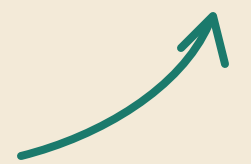
EAGER • DT & SVM

“Dibangun dulu, dipakai berkali-kali.”

- Model klasifikasi / *target function* dibangun dari data latih.
- Training = batch learning penuh, lalu model disimpan.
- Testing cepat: data uji langsung diklasifikasi model.
- SVM memilih hyperplane dengan margin sebesar mungkin.

hasil training = model siap pakai
biaya mahal di depan, murah di belakang

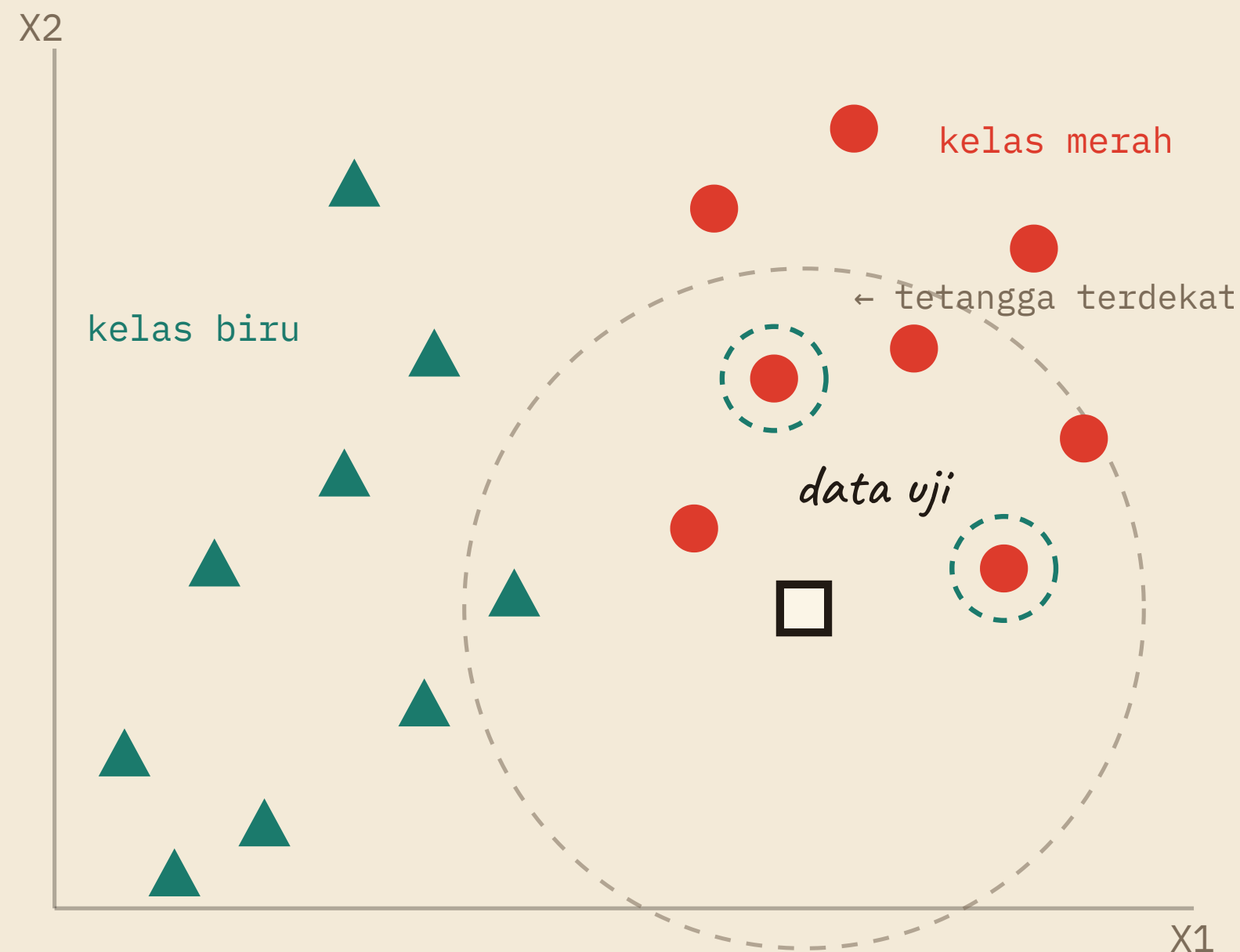
biayanya tidak sama: kNN menunda hitung ke setiap prediksi



INTUISI

YANG TERDEKAT BIASANYA MENANG

Koleksi latih: 8 lingkaran merah dan 9 segitiga biru, dua fitur X_1 dan X_2 . Satu sampel kotak baru harus diprediksi kelasnya.



- Setiap sampel = **feature vector** (X_1, X_2); X_1 jadi sumbu mendatar, X_2 sumbu tegak.
- Tanpa menghitung apa pun, mata langsung menebak kelas **merah**: sampel uji paling dekat dengan lingkaran merah, dan jauh dari semua segitiga biru.
- Idenya: kelas sebuah titik ditentukan oleh tetangga terdekatnya — fitur yang mirip → kelas yang sama.
- Karena itu kNN tidak butuh asumsi distribusi data: cukup data latih dan cara mengukur kemiripan.

intuisi dulu, rumus menyusul — dan data uji di lapangan tidak cuma satu!

CARA KERJA

HITUNG, URUTKAN, VOTING

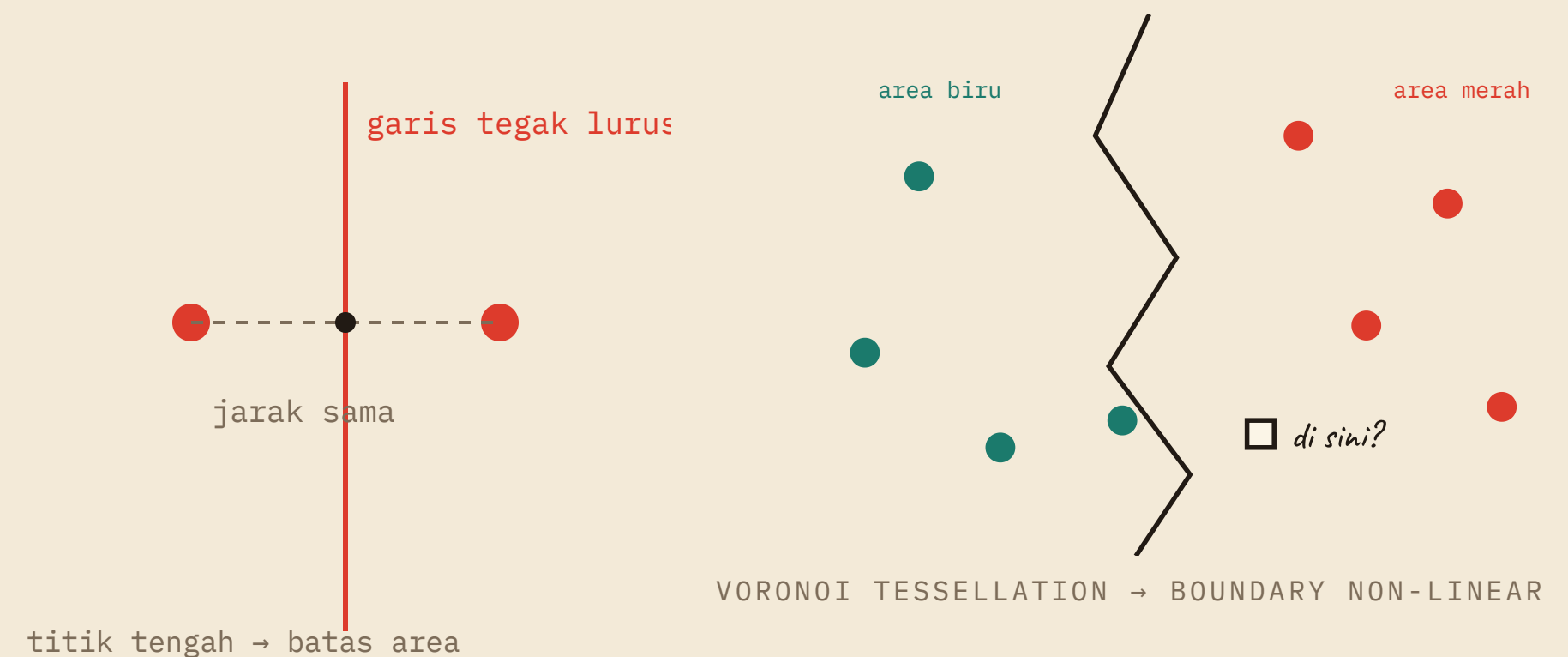
Tiga langkah yang sama dipakai untuk setiap data uji. Batas keputusan akhirnya terbentuk dari area Voronoi tiap titik.

- 1 Hitung jarak**
Ukur kemiripan data uji ke **setiap** sampel latih — misalnya Euclidean distance.
- 2 Urutkan & ambil k**
Cari k sampel dengan jarak **terkecil** (nilai D paling kecil = paling mirip).
- 3 Voting mayoritas**
Lihat label k tetangga itu, lalu ambil kelas yang paling dominan sebagai prediksi.

k = banyaknya tetangga yang dilibatkan — bukan satu-satunya pilihan: coba 1, lalu 3, lalu 5.

CATATAN

Hasil boundary-nya bisa **non-linear** — kNN tidak hanya untuk masalah yang bisa dipisah garis lurus.



BATAS = TITIK YANG BERJARAK SAMA

ALGORITMA

DARI NOTASI KE KEPUTUSAN

Data latih pasangan (X_i, Y_i) ; data uji X tanpa label. Tiga langkah tadi, ditulis formal.

LANGKAH 1 • JARAK

$D(X, X_i)$ untuk setiap i

Ukur kemiripan data uji X dengan tiap sampel latih X_i — pilihan ukurannya adalah hyperparameter.

LANGKAH 2 • k TETANGGA

ambil k nilai D terkecil

Nilai D paling kecil = paling mirip. Ambil sejumlah k sampel latih (X_i, Y_i) dari daftar terurut.

LANGKAH 3 • KEPUTUSAN

$Y = \text{mayoritas}(Y_i \text{ dari } k \text{ tetangga})$

Kelas dengan jumlah suara terbanyak jadi prediksi — bukan jarak totalnya, tapi jumlah suaranya.

EUCLIDEAN DISTANCE

$$D(X, X_i) = \sqrt{\sum_{d=1..D} (X_d - X_{i,d})^2}$$

Dua hyperparameter

- Ukuran kemiripan / distance
- Nilai k (jumlah tetangga)

Keduanya dicoba lewat validation set — bukan dipilih asal.

target function-nya sendiri adalah classification boundary-nya

EUCLIDEAN: SIMETRIS, TAPI PILIH-PILIH SKALA

Ukuran paling umum. Dua sifatnya yang perlu diingat justru yang paling sering bikin prediksi meleset.

Sifat 1 — simetris (spherical)

$D(X, X_i) = D(X_i, X) \rightarrow$ dibolak-balik hasilnya sama

Karena selisihnya dikuadratkan, arah tidak berpengaruh. Semua dimensi juga dianggap setara bobotnya.

Sifat 2 — sensitif perbedaan ekstrem

Fitur dengan rentang besar mendominasi: selisih usia 5 tahun tidak berarti apa-apa di samping selisih gaji 500 ribu.

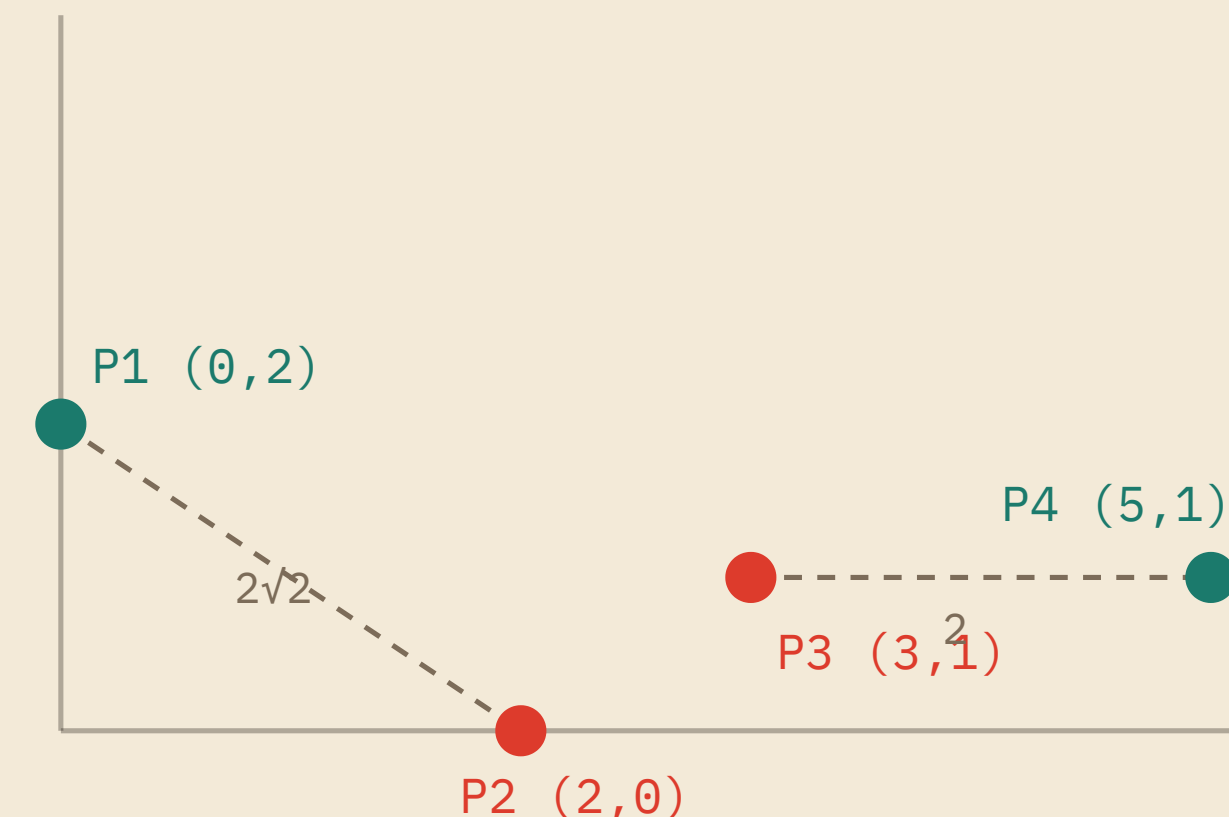
yg mirip: gaji beda 500rb — bukan yang usia beda 5 th
padahal bisa jadi justru usia yang menentukan $\rightarrow$ misclassified

Obatnya: normalisasi (dibahas di halaman isu).

$$D(P1, P2) = \sqrt{(0-2)^2 + (2-0)^2} = \sqrt{8}$$

$$= 2\sqrt{2} \approx 2,83 \quad (P1 = (0,2), P2 = (2,0))$$

$$D(P4, P3) = \sqrt{(5-3)^2 + (1-1)^2} = \sqrt{4} = 2$$



EMPAT ALTERNATIF, SATU RUMUS INDUK

Pilih ukuran sesuai jenis fitur dan bentuk datanya — ini hyperparameter, bukan sekadar detail teknis.

Hamming jumlah fitur yang nilainya berbeda Untuk fitur kategorikal. Suhu tinggi vs rendah = 1; ditambah satu fitur beda lagi = 2. Makin kecil, makin mirip.	Manhattan / city block $\sum_k x_k - y_k$ Selisih tanpa dikuadratkan dan tanpa akar — hanya nilai absolut, lalu dijumlahkan.	Chebyshev / max $\max_k x_k - y_k$ Seperti Manhattan, tapi yang diambil hanya selisih terbesar dari seluruh fitur.
Minkowski $(\sum_k x_k - y_k ^p)^{1/p}$ Rumus umum: $p = 1 \rightarrow$ Manhattan, $p = 2 \rightarrow$ Euclidean, $p \rightarrow \infty \rightarrow$ Chebyshev.	<i>coba satu-satu, bandingkan hasilnya di validation set</i>	Mahalanobis $\sqrt{(x-y)^T \Sigma^{-1} (x-y)}$ Memperhitungkan kovarians antar fitur. Contoh transkrip: dua titik berjarak 14,7 dengan ukuran biasa, ternyata hanya 6 secara Mahalanobis.

INGAT

Semua ukuran sama-sama sah — bedanya cara memandang kemiripan. Pilih yang cocok dengan tipe fitur dan domain datanya.

CONTOH HITUNG • 1/2

KERTAS TISU: HITUNG JARAK KE 4 SAMPEL

Survei kualitas kertas tisu: X1 = daya tahan keasaman, X2 = kekuatan. Kertas baru U = (3, 7) harus diprediksi: kelas T (tinggi) atau R (rendah).

SAMPEL	X1	X2	KELAS
L1	7	7	R
L2	7	4	R
L3	1	4	T
L4	5	10	T
U	3	7	...

urutannya: hitung dulu semua jarak – jangan menebak kelasnya sekarang

$D(U,L1) = \sqrt{(16 + 0)} = \sqrt{16} = 4$	L1 = (7,7)
$D(U,L2) = \sqrt{(16 + 9)} = \sqrt{25} = 5$	L2 = (7,4)
$D(U,L3) = \sqrt{(4 + 9)} = \sqrt{13} \approx 3,61$	L3 = (1,4)
$D(U,L4) = \sqrt{(4 + 9)} = \sqrt{13} \approx 3,61$	L4 = (5,10)

semua jarak siap → lanjut ke urutan & voting

CONTOH HITUNG • 2/2

URUTKAN, LALU VOTING

Jarak diurutkan dari yang terkecil. Hasil prediksi bergantung pada k yang dipilih.

RANK	SAMPEL	D (U , L I)	KELAS
1	L3	3,61	T
2	L4	3,61	T
3	L1	4	R
4	L2	5	R

Ranking = urutan jarak dari terkecil. Voting baru bisa dihitung setelah urutan ini siap.

K = 1 • ONE NEAREST NEIGHBOR

Ranking 1 = L3 (T) → prediksi T

Hanya melihat tetangga paling dekat — paling sensitif terhadap outlier.

K = 3 • VOTING MAYORITAS

2 suara T (L3, L4) vs 1 suara R (L1) → T

Keputusan lebih stabil karena melihat lebih banyak tetangga — yang menang suara, bukan jarak total.

dua nilai k, hasil yang sama — tapi belum tentu selalu begitu!

ISU

PILIH K GANJIL, HINDARI SERI

Untuk klasifikasi biner, k ganjil mencegah seri pada voting sederhana.

● ● ● ● ● k = 5 → 3 vs 2 → kelas teal menang

● ● ● ● k = 4 → 2 vs 2 → seri! ?

Keputusan seri bukan pilihan — harus ada jalan keluar:

1 • Undi / acak

Ibarat melempar koin untuk memilih kelas pemenang.

2 • Turun ke 1-NN

Pakai tetangga terdekat saja — selalu menghasilkan satu kelas.

3 • Pakai prior knowledge

Lihat kelas mana yang lebih dominan di keseluruhan data latih.

kNN dasar memberi label, bukan confidence terkalibrasi; voting lokal kurang peka pada prior kelas

ISU

DATA BOLONG, JARAK TAK BISA DIHITUNG

Satu nilai atribut kosong (null) membuat $D(X, X_i)$ tidak terdefinisi. Selesaikan dulu, baru hitung kemiripan.

SAMPEL	SUHU BADAN	GEJALA?	KELAS
S1	38,2	ya	A
S2	?	ya	B
S3	36,9	tidak	A
S4	37,8	tidak	B

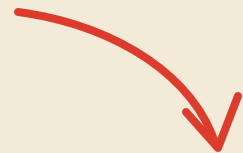
SOLUSI PALING SEDERHANA

Isi dengan rata-rata atributnya

$? \rightarrow (38,2 + 36,9 + 37,8) / 3 = 37,63$

Setelah terisi, barulah jarak kemiripan bisa dihitung. Nilai imputasi lain (median, model) juga boleh — pilih yang paling masuk akal.

missing value bukan cuma masalah kNN — strategi ini dipakai di metode lain juga



selama ada ? di data, kemiripan tidak bisa dipercaya

ISU

SAMAKAN SKALA SEBELUM MENGHITUNG JARAK

Fitur dengan rentang besar menelan fitur kecil. Dua teknik yang dibahas di transkrip:

ZERO MEAN UNIT VARIANCE

$$x'_{im} = (x_{im} - \mu_m) / \sigma_m$$

MIN-MAX NORMALIZATION

$$x' = (x - x_{min}) / (x_{max} - x_{min})$$

Min-max memaksa semua nilai masuk rentang **0 sampai 1**. Ini normalisasi versi machine learning — beda dengan normalisasi di basis data.

latih: cek dulu rentang tiap fitur sebelum percaya hasil kNN

Kenapa perlu? Lihat usia vs gaji.

ORANG	USIA	GAJI
A	25	3.000.000
B	45	8.000.000

selisih usia = 20

selisih gaji = 5.000.000 — mendominasi total jarak

Akibatnya kemiripan jadi bias ke fitur bergaji — bisa salah prediksi.

ISU

K TERBAIK? COBA SATU-SATU

Besar k memengaruhi kinerja: tidak ada rumus ajaib, ada prosedur uji coba.

k terlalu kecil

Boundary tidak stabil: satu outlier bisa mengubah batas keputusan secara drastis.

$k = 1$ • paling sensitif

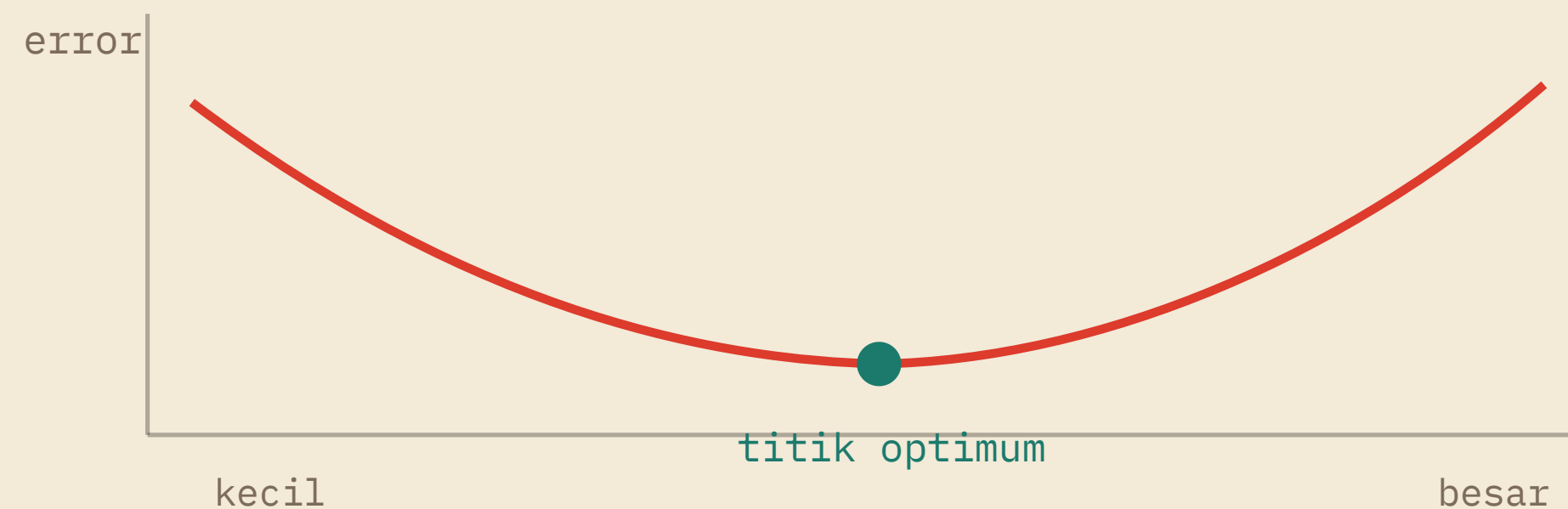
k terlalu besar

Prediksi condong ke kelas yang dominan di data latih — detail lokal hilang.

$k \rightarrow N$ • selalu kelas mayoritas

Prosedur

Ambil validation set
Uji $k = 1, 3, 5, \dots$
Catat kinerjanya
Berhenti saat pola turun



uji coba itu bagian dari pekerjaan — bukan tanda tidak paham

ISU

BIAYA KOMPUTASI & JALAN PINTASNYA

Setiap data uji dibandingkan dengan seluruh data latih — mahal, apalagi kalau dimensinya besar.

BIAYA PER PREDIKSI

$O(N \cdot D)$

N = jumlah sampel latih, D = jumlah fitur. Training-nya murah (hanya menyimpan feature vector), testing-nya yang berat.

10 data uji × 100 latih = 1.000 × hitung jarak

Kurangi D (kolom)

Dimensionality reduction atau pemilihan fitur.

Kurangi N (baris)

Ambil subset sampel representatif — strategi pemilihannya juga tantangan.

KD-tree — pintasan pencarian tetangga

$\approx O(\log_2 N)$ rata-rata*

Risikonya: tetangga yang sebenarnya paling mirip bisa jadi tidak ikut diperiksa.

*Dapat memburuk pada dimensi tinggi.

Inverted list

Umum untuk data teks.

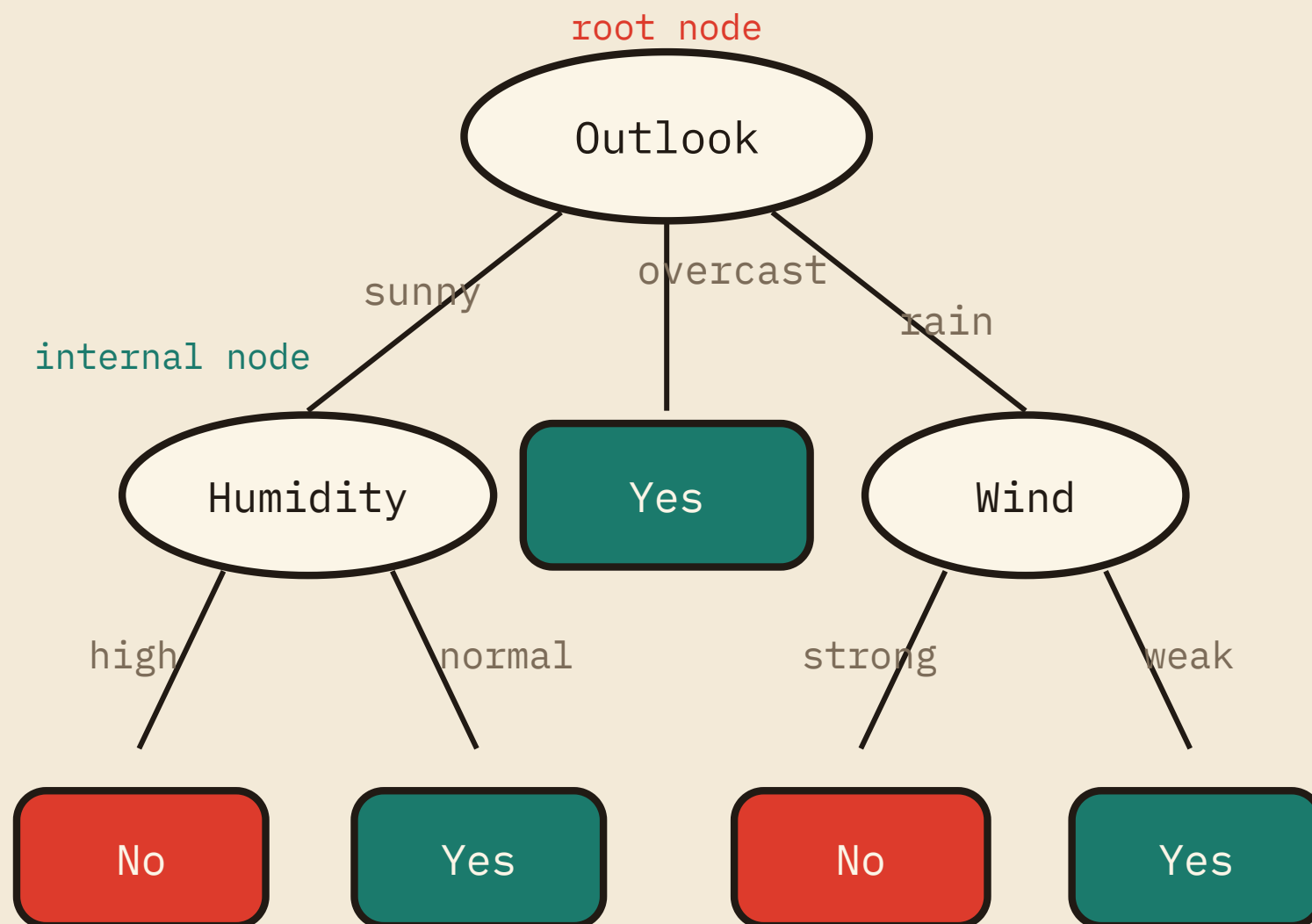
Locality-sensitive hashing

Dipakai di fingerprinting.

DEFINISI

ANATOMI POHON KEPUTUSAN

Struktur pohon untuk menspesifikasikan rangkaian pengambilan keputusan dan konsekuensinya.



rounded rect = leaf node (kelas)

depth = 1 • dari root ke internal node terjauh

Node = satu fitur / kriteria

Node paling atas disebut **root**; node keputusan di bawahnya **internal node**.

Branch = kemungkinan nilai

Bisa biner (laki-laki/perempuan) atau lebih dari dua; untuk data kontinu berupa rentang.

Leaf = kelas hasil prediksi

Digambar sebagai rounded rectangle — di sinilah keputusan berakhir.

Depth = kedalaman pohon

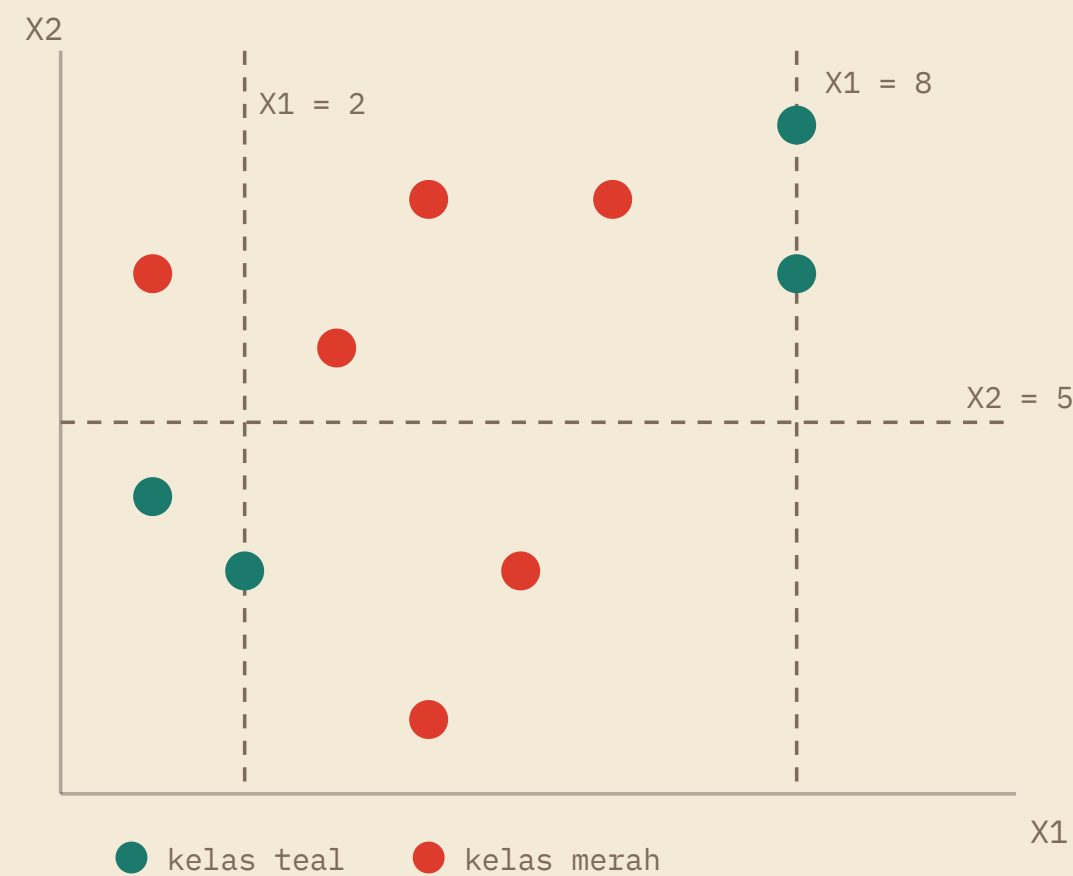
Dihitung dari root ke internal node terjauh — makin dangkal, makin efisien.

notasi boleh luwes: cabang bisa ditulis langsung di garis, leaf pakai kotak biasa

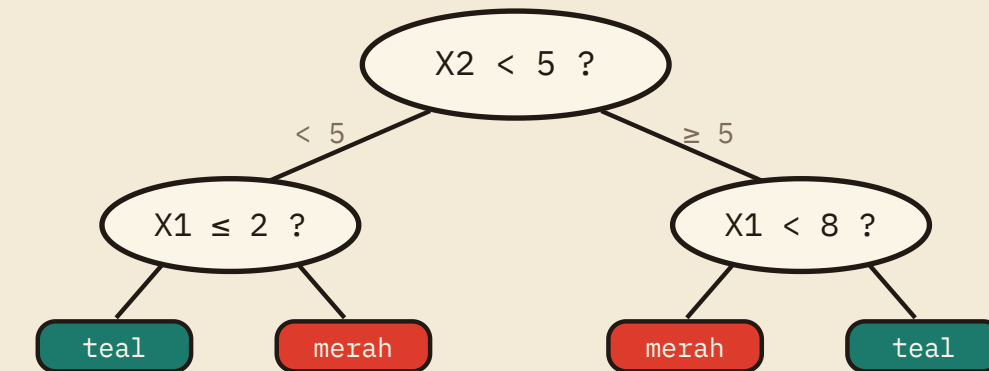
DEFINISI • LANJUTAN

SATU DATA, DUA POHON

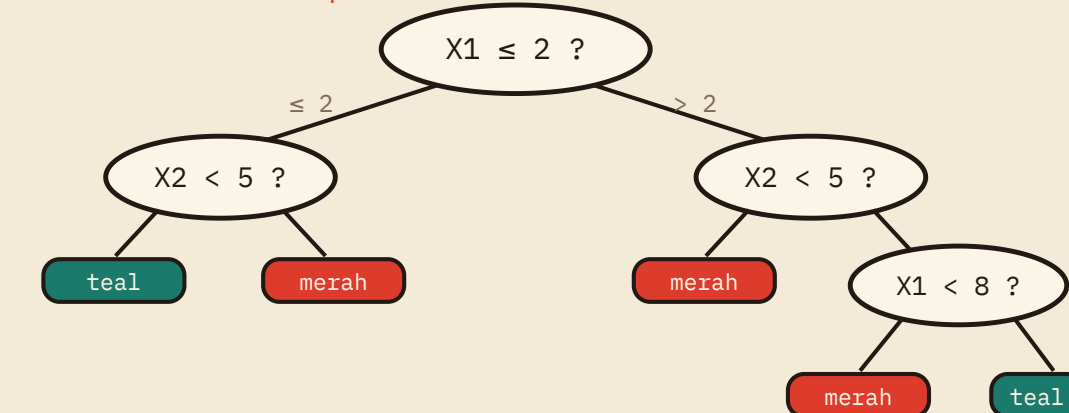
Dua kelas, dua fitur (X_1 , X_2). Pohon yang lebih dangkal lebih efisien — itulah yang dicari algoritma saat memilih atribut split.



POHON A • root X_2 • depth 1



POHON B • root X_1 • depth 2



KESIMPULAN

Pohon A lebih efektif: depth 1 vs depth 2 — splitting berikutnya tidak perlu banyak.

STUDI KASUS

14 HARI MAIN TENIS

Data latih: 14 sampel, 3 fitur (outlook, humidity, wind) + 1 kelas (9 yes, 5 no). Data uji baru: rain, high, weak — main atau tidak?

HARI	OUTLOOK	HUM	WIND	PLAY
1	Sunny	High	Weak	No
2	Sunny	High	Strong	No
3	Overcast	High	Weak	Yes
4	Rain	High	Weak	Yes
5	Rain	Normal	Weak	Yes
6	Rain	Normal	Strong	No
7	Overcast	Normal	Strong	Yes

HARI	OUTLOOK	HUM	WIND	PLAY
8	Sunny	High	Weak	No
9	Sunny	Normal	Weak	Yes
10	Rain	Normal	Weak	Yes
11	Sunny	Normal	Strong	Yes
12	Overcast	High	Strong	Yes
13	Overcast	Normal	Weak	Yes
14	Rain	High	Strong	No

DATA UJI

outlook = rain · humidity = high · wind = weak
Ikuti cabangnya sampai daun — keputusan apa yang keluar?

Kolom “**Hari**” hanya nomor urut — tidak dipakai saat training maupun testing.

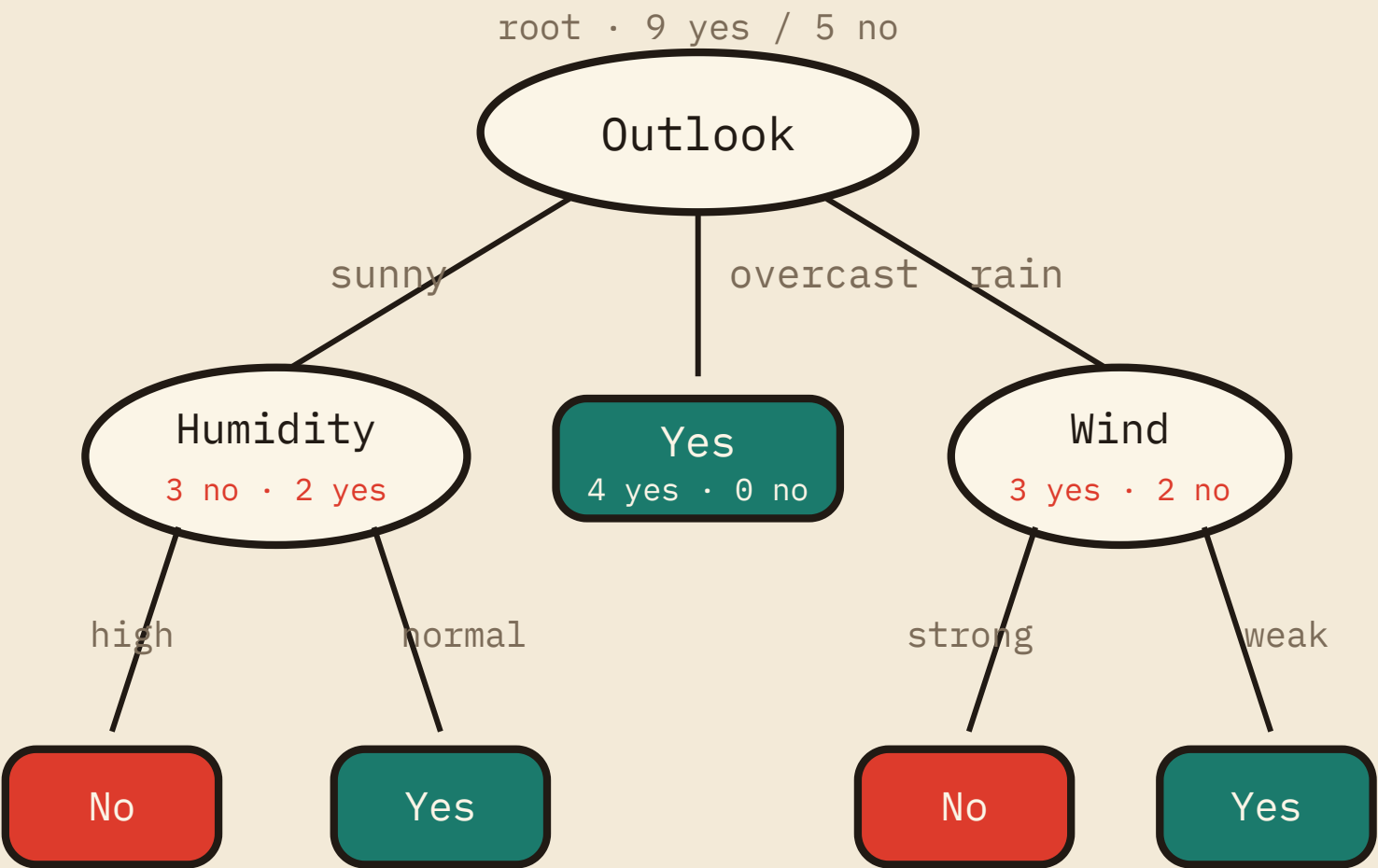
perhatikan komposisinya: 9 yes, 5 no — dari sini semua perhitungan dimulai

CARA KERJA

PECAH TERUS SAMPAI MURNI

Satu fitur diuji, cabang yang sudah satu kelas berhenti. Cabang yang masih campur dipecah lagi dengan fitur lain.

outlook = overcast → berhenti, daun = Yes	4 yes • 0 no → pure
outlook = sunny → pecah lagi dengan humidity	3 no • 2 yes → belum pure
sunny + humidity high → No	high → 3 no (pure)
sunny + humidity normal → Yes	normal → 2 yes (pure)
outlook = rain → pecah lagi dengan wind	3 yes • 2 no → belum pure
rain + wind → dua daun pure	strong → No • weak → Yes

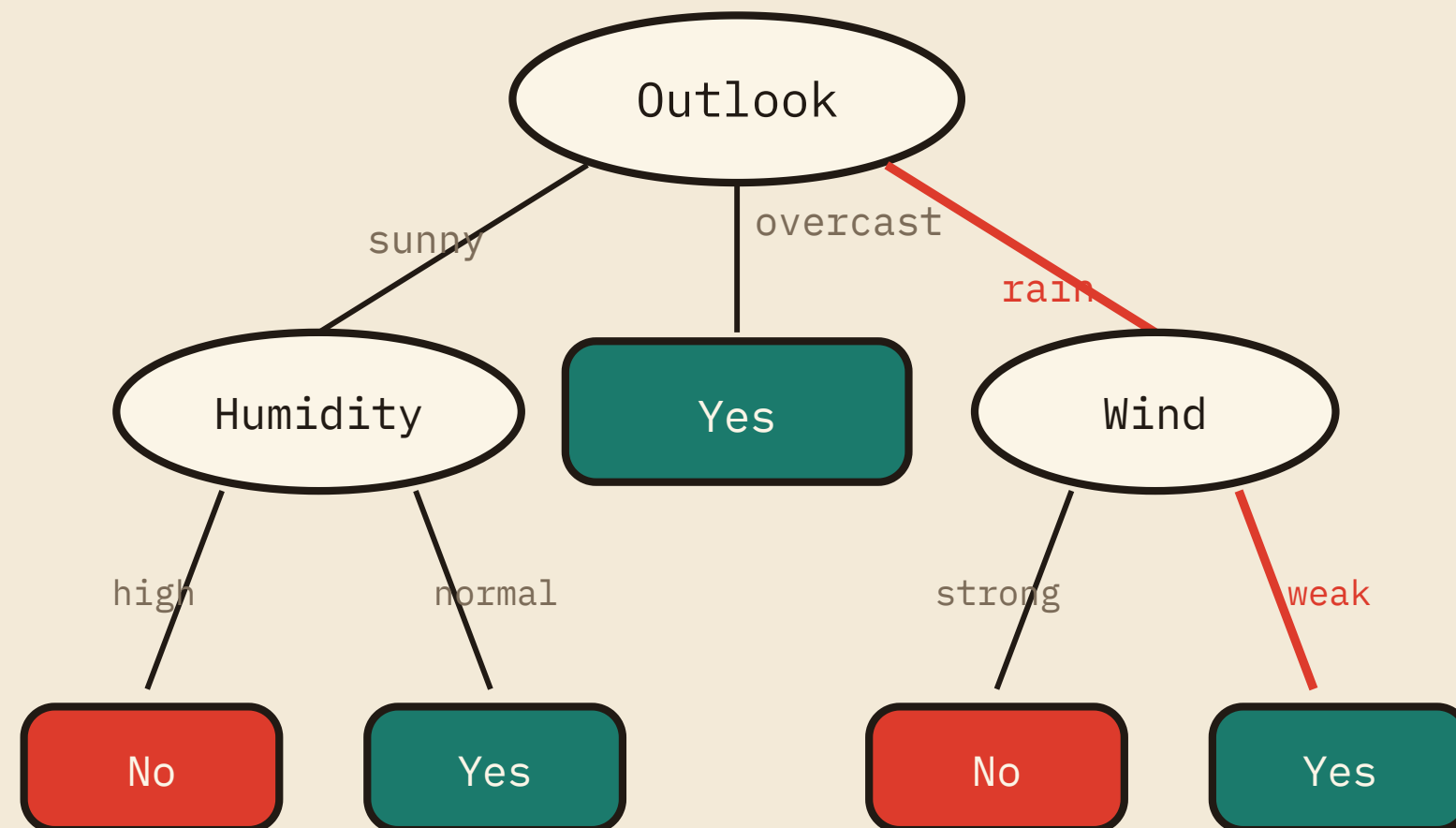


- aturan berhenti:
- 1. subset sudah pure (satu kelas)
 - 2. atribut sudah habis dipakai

HASIL AKHIR

POHON FINAL & SATU PREDIKSI

Pohon lengkap dari 14 sampel. Sekarang jalankan data uji (rain, high, weak) dari root sampai daun.



garis tebal = jalur data uji

1 • outlook = rain

dari root, ikuti cabang kanan menuju node Wind.

2 • wind = weak

humidity tidak dicek di cabang ini — pohon hanya menguji wind.

3 • prediksi = Yes

sampel uji diklasifikasi “bermain tenis”.

perhatikan: humidity-nya high, tapi tidak pernah dilihat pohon

UKURAN

ENTROPI: DERAJAT KETIDAKPASTIAN

Kenapa perlu ukuran? Karena dengan ribuan sampel, mata tidak bisa lagi menilai split mana yang paling “bersih”.

ENTROPY

$$H(S) = - \sum_{c \in C} p_c \log_2 p_c$$

p_c = probabilitas kemunculan kelas c di dalam subset S . Skalanya: **0 = pasti, 1 = paling tidak pasti**.

Pure set → H = 0
4 yes / 0 no · 0 yes / 4 no · n yes / 0 no
Semua kelasnya sama; tidak ada ketidakpastian. (Probabilitasnya bisa 100% atau 0% — dua-duanya “pasti”).

Impure set 50:50 → H = 1
3 yes / 3 no · 4 / 4 · 5 / 5 · dst.
Paling tidak pasti — karena itu nilainya maksimal.

$p_{\text{yes}} = 3/6 = 0,5 \cdot p_{\text{no}} = 3/6 = 0,5$	komposisi 3 : 3
$H(S) = -(0,5 \times -1) - (0,5 \times -1)$	$\log_2 0,5 = -1$
$H(S) = 0,5 + 0,5 = 1$	50:50 selalu begitu
$H(S) = -(1 \times \log_2 1) - (0 \times \log_2 0) = 0$	4 yes / 0 no
$H(9 \text{ yes}, 5 \text{ no}) = -(9/14 \log_2 9/14) - (5/14 \log_2 5/14) \approx 0,94$	bekal untuk halaman berikutnya

UKURAN

INFORMATION GAIN: PILIH SPLIT TERBAIK

Gain = penurunan ketidakpastian setelah data dipecah dengan sebuah fitur. Fitur dengan gain tertinggi jadi root.

INFORMATION GAIN

$$IG(S, F) = H(S) - \sum_{f \in F} \left(\frac{|S_f|}{|S|} \right) H(S_f)$$

H(S) = 0,94

14 sampel

wind: 0,94 - (8/14 × 0,811) - (6/14 × 1)

weak: 8 sampel, H = 0,811

IG(wind) = 0,048

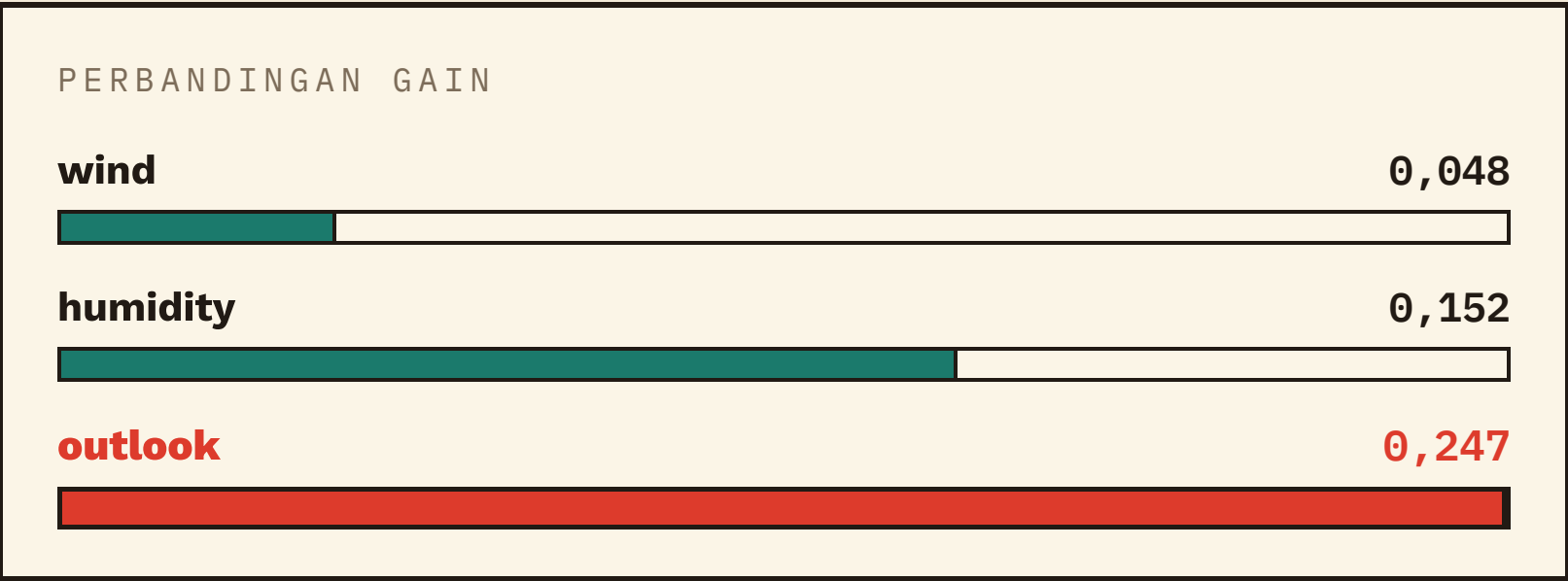
strong: 6 sampel (3:3), H = 1

IG(humidity) = 0,152

dihitung dengan cara sama

IG(outlook) = 0,247

tertinggi → root!



Outlook punya satu cabang yang langsung pure (overcast = 4 yes / 0 no) — itulah kenapa ia paling informatif.

proses diulang di tiap cabang sampai semua daun pure

UKURAN • LANJUTAN

ULANGI DI SETIAP CABANG

Setelah root terpilih, perhitungan yang sama dijalankan pada tiap cabang — dengan subset data dan fitur yang tersisa.

CABANG SUNNY • 5 SAMPEL (2 YES, 3 NO)

Fitur **outlook** tidak dipertimbangkan lagi di cabang ini — sudah dipakai di atasnya.

$IG(\text{humidity}) = 0,971$ high → 3 no (pure) • normal → 2 yes (pure)

$IG(\text{wind}) = 0,020$ kalah jauh

pemenang: **humidity** → internal node berikutnya

CABANG RAIN • 5 SAMPEL (3 YES, 2 NO)

Dengan sisa fitur yang ada, satu fitur langsung memisahkan keduanya dengan sempurna.

$IG(\text{wind}) = 0,971$ strong → No • weak → Yes (dua-duanya pure)

$IG(\text{humidity}) = 0,020$ kalah jauh

pemenang: **wind** → internal node berikutnya

HASIL

Root outlook → sunny memakai humidity → rain memakai wind — persis pohon yang sudah kita prediksi di halaman sebelumnya.

PENERAPAN

DARI POHON KE SET OF RULES

Model pohon diubah menjadi kumpulan aturan — satu aturan untuk tiap daun, dibaca dari root menyusuri cabang.

CONJUNCTION — SEMUA SYARAT HARUS TERPENUHI

IF outlook=sunny AND humidity=high
THEN play = No

DISJUNCTION — CUKUP SALAH SATU

IF outlook=overcast OR outlook=rain
THEN play = Yes

Contoh OR di atas berlaku untuk bentuk pohon yang berbeda — kalau dua cabang menghasilkan daun yang sama, cukup digabung dengan “atau”.

satu daun = satu aturan; aturan dibangun per leaf, bukan per cabang

SET OF RULES — POHON MAIN TENIS

- 1 IF outlook=sunny AND humidity=high → No
- 2 IF outlook=sunny AND humidity=normal → Yes
- 3 IF outlook=overcast → Yes
- 4 IF outlook=rain AND wind=strong → No
- 5 IF outlook=rain AND wind=weak → Yes

5 daun → 5 aturan. Aturan ke-3 cukup satu syarat karena jalurnya pendek.

KINERJA

SATU NOISE, POHON BERTAMBAH DALAM

Pohon yang dipaksa cocok dengan data aneh justru kehilangan kemampuan generalisasi — itulah overfitting.

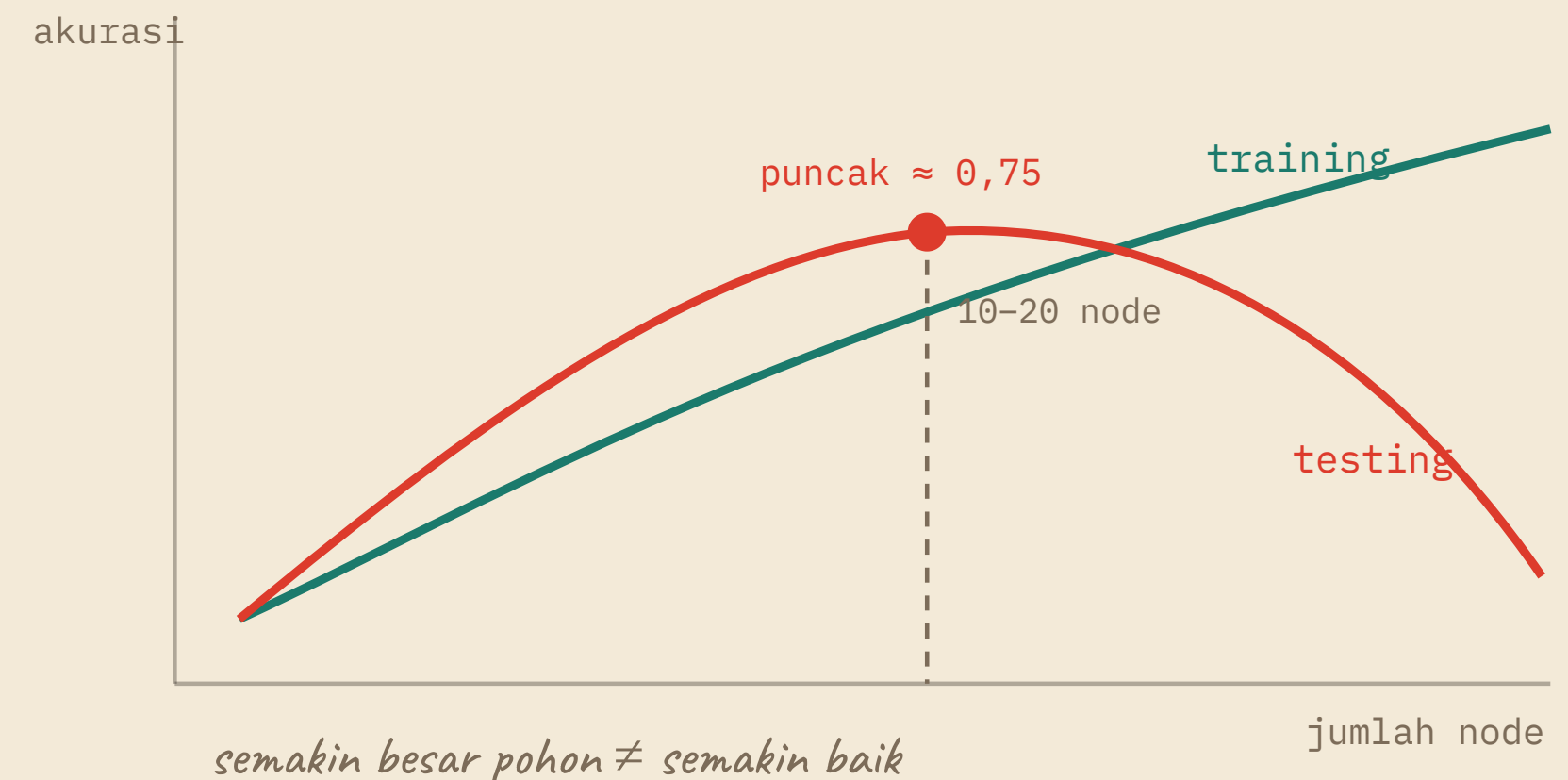
CONTOH NOISE

sampel baru: sunny • normal • strong

keputusan aktual: **No**

Padahal pohon yang ada memprediksi **Yes** (humidity normal). Pohon harus tumbuh lebih dalam hanya untuk satu titik ini.

- Akibatnya: cabang tambahan yang sangat spesifik pada noise.
- Data uji normal lain justru ikut salah klasifikasi.
- Overfitting juga muncul kalau data latih belum representatif (tidak mencakup seluruh kemungkinan).



Akurasi data latih terus naik; akurasi data uji justru turun setelah titik optimum. Yang kita pedulikan adalah yang kedua.

STRATEGI

STOP AWAL ATAU PANGKAS BELAKANGAN

Dua pendekatan menahan pohon agar tidak terlalu kompleks — keduanya butuh satu komponen baru: validation set.

PENDEKATAN 1

Early stopping

Hentikan proses growing the tree **seawal mungkin**: begitu split baru tidak lagi menambah manfaat, berhenti.

node tidak ditumbuhkan → tidak perlu dipotong

PENDEKATAN 2

Pruning

Biarkan pohon tumbuh penuh dulu, lalu **potong** cabang / internal node yang hanya cocok untuk noise.

tumbuh dulu → pangkas setelah evaluasi

Kunci: validation set — bukan training, bukan testing

Training

Validation

Testing

Untuk setiap kandidat split / cabang: uji ke validation set. Kalau akurasi **menurun** — di internal node itu, berhenti.

VARIAN

C4.5: MENAMBAL BIAS ID3

ID3 cenderung memilih fitur dengan banyak kemungkinan nilai — gain-nya tinggi semu. C4.5 menormalkannya dengan gain ratio.

Masalahnya

- Fitur dengan nilai sangat banyak (diskret maupun kontinu) disukai ID3.
- Padahal belum tentu fitur itu paling berguna.
- Contoh: fitur angin dengan 2 nilai aman; fitur dengan puluhan nilai menipu.

GAIN RATIO

$$\text{GainRatio}(S, A) = \text{Gain}(S, A) / \text{SplitInfo}(S, A)$$

SPLIT INFORMATION

$$\text{SplitInfo}(S, A) = - \sum_{i=1..c} (S_i / S) \log_2 (S_i / S)$$

S_i adalah jumlah kasus untuk masing-masing nilai dari fitur A. Split info “menghukum” fitur yang memecah data jadi banyak bagian.

Yang berubah hanya ukuran split-nya

Alur pohon, cara membaca aturan, dan masalah overfitting-nya sama dengan ID3 — yang diganti adalah kriteria pemilihan atribut.

C4.5 bukan satu-satunya varian, tapi paling umum dipakai

ISU

KALAU ATRIBUTNYA BERUPA ANGKA

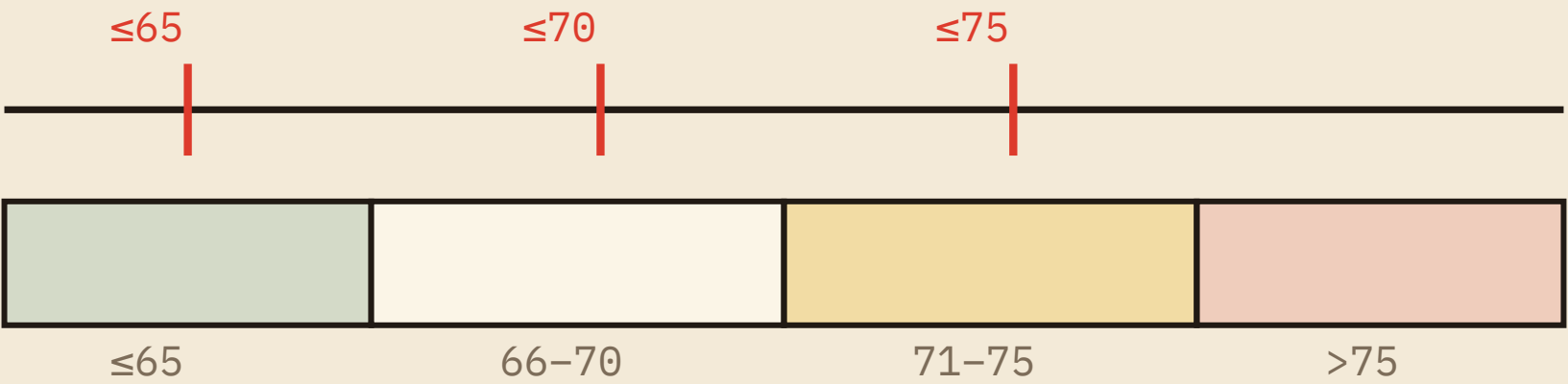
Humidity yang tadinya high/normal bisa datang sebagai angka. Split tidak lagi per kategori, tapi per interval.

urutkan semua nilai humidity dari kecil ke besar	langkah 1
ambil distinct value – 70 yang muncul 3× cukup ditulis sekali	langkah 2
bentuk interval: ≤ 65 / > 65 , lalu ≤ 70 / > 70 , dan seterusnya	langkah 3
hitung gain tiap interval, pilih yang tertinggi	langkah 4

prosesnya sama seperti fitur kategorikal – cuma batasnya sekarang berupa angka



NILAI TERURUT → DISTINCT VALUE



tiap kandidat batas dievaluasi gain-nya – ambil yang terbaik
berbeda dari kNN: di sini nilai jadi batas split

ISU

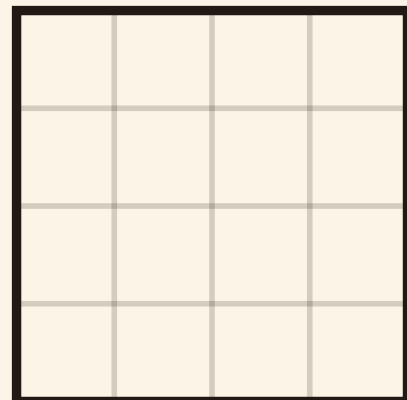
DARI DATA MENTAH KE VEKTOR FITUR

Sebagian besar algoritma pembelajaran butuh data dalam representasi numerik. Tiga kasus yang muncul di transkrip:

NUMERIK

Gambar → vektor

Citra 28×28 piksel diperlakukan sebagai vektor
784 nilai intensitas.



$[X_1, X_2, \dots, X_{784}]$

TEKS

Dokumen → frekuensi

Email atau dokumen direpresentasikan lewat
frekuensi kemunculan kata — ada banyak varian lain.

"rapat": 3 · "nilai": 7
"ujian": 1 · ...

KATEGORIKAL

Dua nilai → 0/1

Fitur dengan tepat dua kemungkinan paling mudah: petakan langsung ke 0 dan 1.

yes → 1 · no → 0
high → 1 · normal → 0

Tapi bagaimana kalau nilainya lebih dari dua? → halaman berikutnya.

CATATAN

Untuk fitur numerik real, nilainya bisa dipakai langsung tanpa transformasi khusus.

ISU

FITUR BERNILAI LEBIH DARI DUA

Memetakan sunny=0, overcast=1, rain=2 berbahaya: angkanya menyiratkan jarak dan urutan yang tidak nyata.

Masalah jarak palsu

Warna red=0, blue=1, green=2: seolah red–blue “berjarak” sama dengan blue–green, dan red lebih dekat ke blue daripada ke green. Padahal tidak begitu.

Solusi: one-hot encoding

Setiap nilai fitur menjadi **fitur biner baru**: muncul (1) atau tidak (0).

outlook (3 nilai) → 3 fitur baru

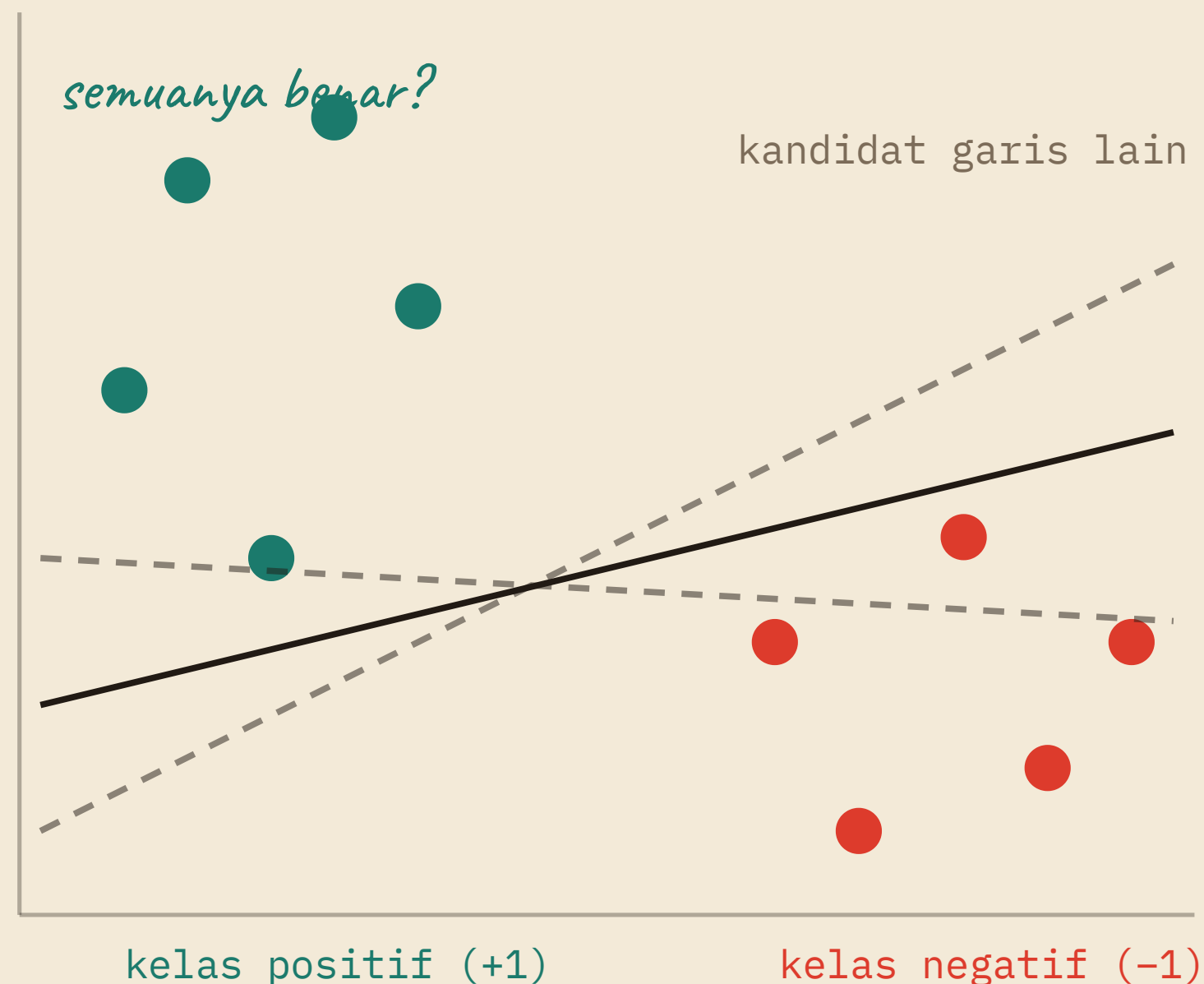
SAMPEL	OUTLOOK	F_SUNNY	F_OVERCAST	F_RAIN
1	Sunny	1	0	0
2	Sunny	1	0	0
3	Overcast	0	1	0
4	Rain	0	0	1
5	Rain	0	0	1

tidak hanya untuk decision tree – berlaku di supervised learning lain

INTUISI

BANYAK GARIS BENAR, SATU PALING BAIK

Pengklasifikasi linear: $f(x) = \text{sign}(w \cdot x + b)$. Dua kelas bisa dipisah dengan tak hingga garis — mana yang dipilih?



- **Signum**: nilai positif $\rightarrow$ kelas +1; negatif $\rightarrow$ -1; nol tepat di hyperplane.
- Semua garis pada gambar sama-sama mengklasifikasikan data latih dengan benar — tapi belum tentu benar untuk data uji baru.
- “Paling baik” = memisahkan data latih dengan margin maksimum; harapannya, batas ini juga menggeneralisasi ke data baru.
- Garis pemisah terbaik disebut **hyperplane** — di ruang 2D ia berupa garis, di dimensi lebih tinggi berupa bidang.

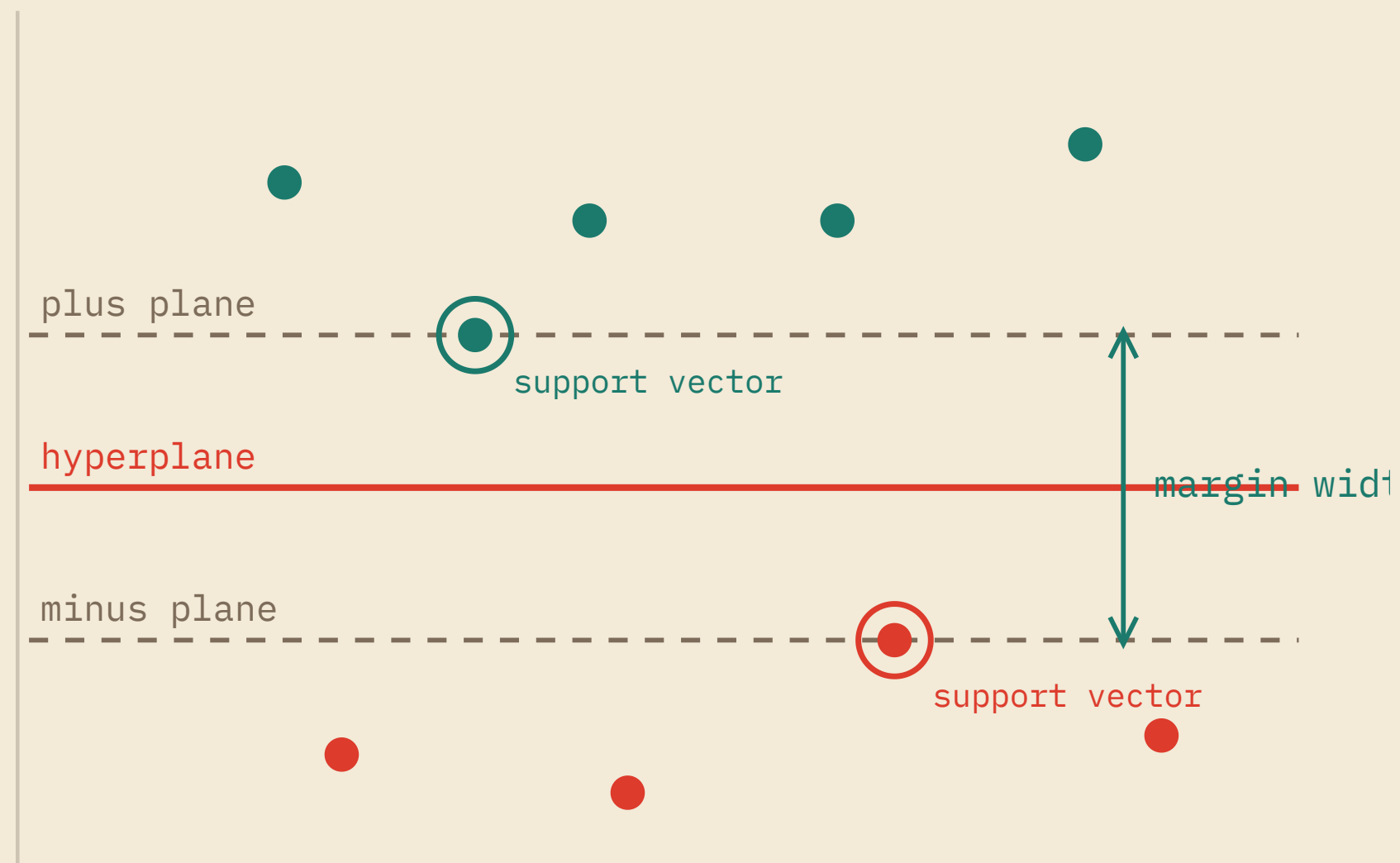
Cara memilihnya: ukur **margin** tiap kandidat, lalu ambil yang **terlebar**. Itulah Linear SVM (LSVM).

cari hyperplane dengan margin maksimum

KONSEP

MARGIN MAKSIMUM & SUPPORT VECTOR

Support vector berada pada margin plane; lebar pita margin adalah dua kali jarak hyperplane ke support vector.



Kenapa margin harus maksimal?

Margin lebar = ruang aman lebih besar → kemungkinan data uji baru salah klasifikasi mengecil.

Hanya support vector yang menentukan

Titik pada margin (support vector) menentukan hyperplane; titik yang jauh dari margin biasanya tidak mengubah solusinya.

Hasil empiris

Kombinasi **maximum margin** + **support vector** terbukti memberi hasil sangat baik pada banyak uji coba.

titik yang jauh dari garis tidak ikut “menyokong” keputusan

TIGA GARIS, SATU ATURAN

Untuk mendapatkan hyperplane, kita spesifikasikan tiga garis: hyperplane itu sendiri, plus plane, dan minus plane.

PLUS PLANE — MELALUI SUPPORT VECTOR KELAS +1

$$w \cdot x + b = +1$$

HYPERPLANE — YANG DICARI

$$w \cdot x + b = 0$$

MINUS PLANE — KELAS -1

$$w \cdot x + b = -1$$

Aturan klasifikasi

$$w \cdot x + b \geq +1 \rightarrow \text{kelas } +1$$

$$w \cdot x + b \leq -1 \rightarrow \text{kelas } -1$$

$$-1 < w \cdot x + b < +1 \rightarrow \text{area margin}$$

Fakta geometris: w tegak lurus bidang

Ambil dua vektor U dan V di plus plane: $W \cdot (U - V) = 0$ — hasil kali titiknya nol, artinya W **ortogonal** terhadap bidang.

Titik pada minus plane ditulis x^- , titik terdekat di plus plane ditulis x^+ . Keduanya terhubung arah W — bekal untuk menghitung margin.

margin bukan jarak antar kelas — tapi lebar area di antara dua plane

DARI DUA PLANE KE $M = 2 / \sqrt{(W \cdot W)}$

Empat baris aljabar yang mengubah “margin selebar mungkin” menjadi fungsi objektif yang bisa dioptimalkan.

$x^+ = x^- + \lambda W$	bergerak dari x^- ke x^+ searah W
$(w \cdot x^- + b) + \lambda(w \cdot w) = +1$	substitusi ke persamaan plus plane
$-1 + \lambda \ w\ ^2 = 1 \rightarrow \lambda = 2 / (w \cdot w)$	karena $w \cdot x^- + b = -1$
$M = \lambda \ W\ = 2 / \sqrt{(w \cdot w)}$	panjang vektor = $\sqrt{(w \cdot w)}$
maksimalkan $2 / \sqrt{(w \cdot w)}$	margin maksimum
$\equiv$ minimalkan $\frac{1}{2} \ w\ ^2$ dengan kendala $y_i (w \cdot x_i + b) \geq 1$	bentuk yang dioptimalkan

FUNGSI OBJEKTIF

$\min \frac{1}{2} \|w\|^2$

Solusinya lewat quadratic programming

Masalah optimasi dengan kendala pertidaksamaan ini diselesaikan dengan teknik QP — di deck ini kita cukup memahami bentuk objektifnya.

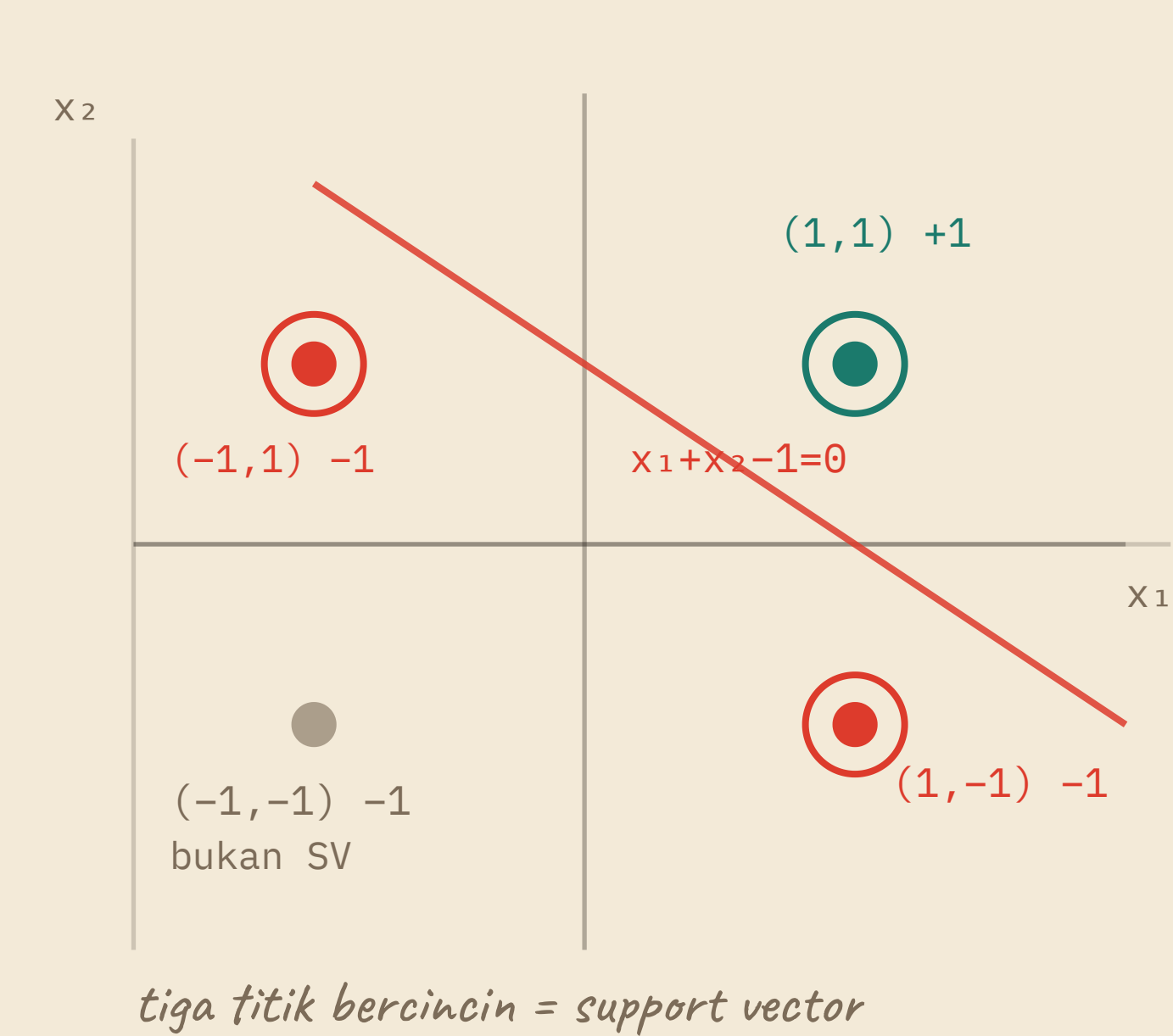
$\|w\|$ adalah panjang vektor W . Semakin kecil $\|w\|$, semakin besar margin $2/\|w\|$ — karena itu keduanya ekuivalen.

$\frac{1}{2}$ dipakai supaya turunannya nanti rapi — tidak mengubah solusinya

CONTOH • 1/2

EMPAT TITIK, TIGA SUPPORT VECTOR

Data minimal: dua fitur, kelas +1 dan -1. Tiga titik menjadi support vector; satu titik tidak berpengaruh.



$y_i (w \cdot x_i + b) \geq 1$ untuk $i = 1 \dots 4$ **4 kendala**

kendala 1 • kelas +1 **$(+1, 1): w_1 + w_2 + b \geq 1$**

kendala 3 • titik $(-1, 1)$ **$(-1, 1): w_1 - w_2 - b \geq 1$**

eliminasi: $w_2 = 1$ **kendala 1 + 2 $\rightarrow w_2 = 1$**

eliminasi: $w_1 = 1$ **kendala 1 + 3 $\rightarrow w_1 = 1$**

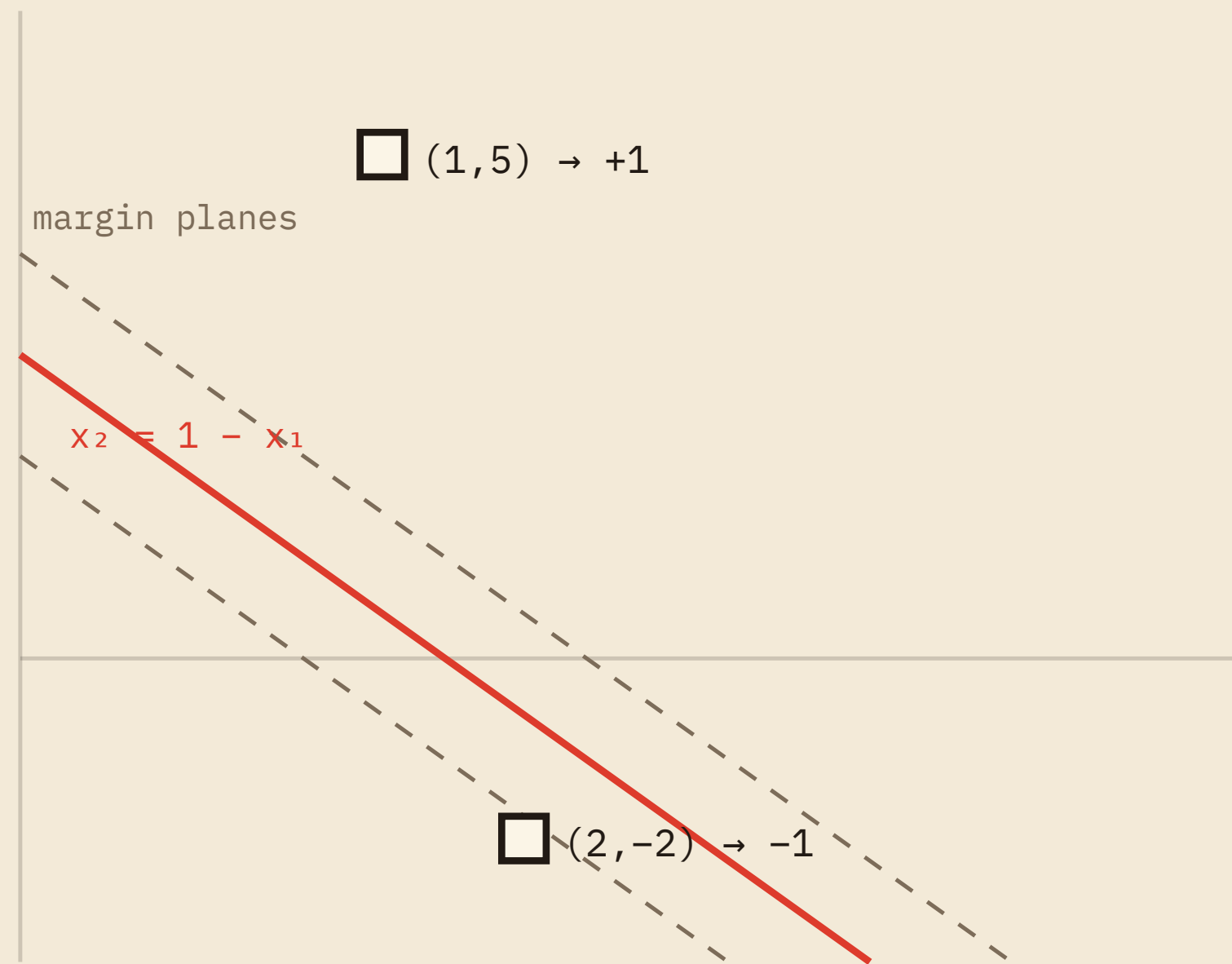
eliminasi: $b = -1$ **kendala 2 + 3 $\rightarrow -2b = 2$**

$x_1 + x_2 - 1 = 0 \rightarrow x_2 = 1 - x_1$ **hyperplane final**

CONTOH • 2/2

UJI HYPERPLANE KE DUA TITIK BARU

Dengan $x_1 + x_2 - 1 = 0$, prediksi cukup dihitung lewat tanda fungsi $f(x)$.



$f(1,5) = +1$ (signum positif) $1 + 5 - 1 = 5 > 0$

$f(2,-2) = -1$ (signum negatif) $2 - 2 - 1 = -1 < 0$

Cara membaca hasilnya

Substitusi nilai x_1 dan x_2 ke persamaan hyperplane, lalu ambil tandanya.
Untuk titik lain, konsepnya sama.

Titik (1,5) berada jauh di sisi kelas +1 — hasilnya konsisten dengan gambar.
Begitu juga (2,-2) di sisi kelas -1.

prediksi SVM = tanda fungsi, bukan jarak

NON-LINEAR

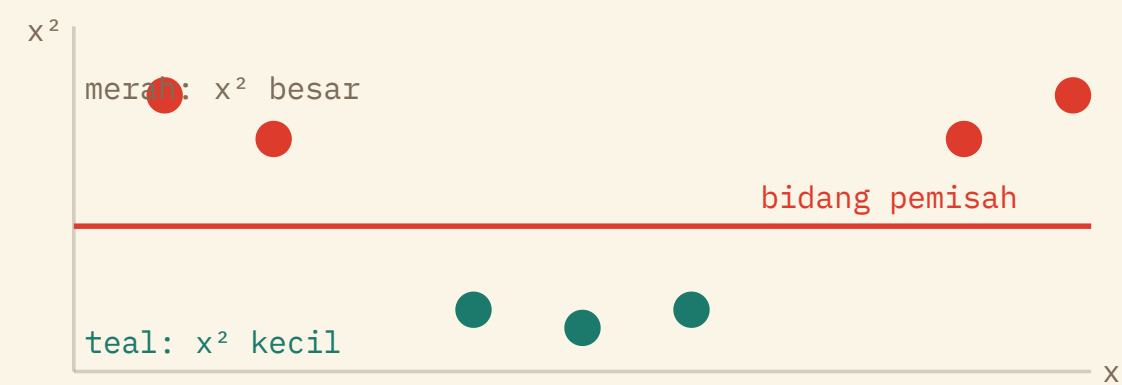
KALAU SATU GARIS TIDAK CUKUP

Data yang tidak terpisah linear dipetakan ke ruang berdimensi lebih tinggi — di sana garis lurus kembali bekerja.

MASALAH • SATU DIMENSI



SOLUSI • DUA DIMENSI



Kernel trick

Pemetaan ke dimensi lebih tinggi dilakukan lewat **fungsi kernel** — tanpa menghitung transformasi eksplisit. Fungsi linear di ruang baru = classifier non-linear di ruang asal.

JENIS KERNEL

polinomial $(x^T y + c)^d$
RBF $\exp(-\|x-y\|^2 / 2\sigma^2)$
sigmoid $\tanh(\alpha x^T y + c)$

Di ruang 3D, hyperplane-nya berupa **bidang**. Semakin kompleks data, semakin banyak varian kernel yang dikembangkan peneliti.

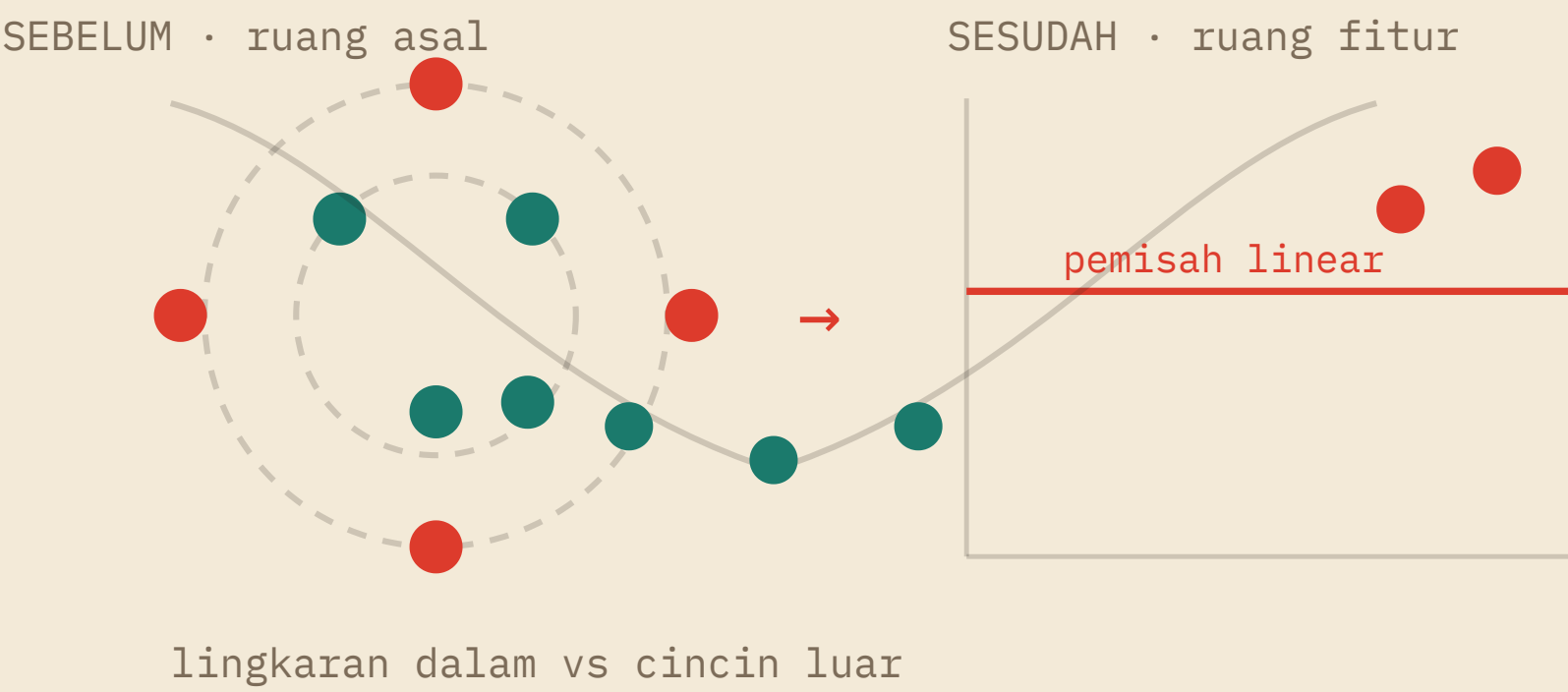
Linear kernel → data terpisah garis; kernel non-linear → data yang tidak terpisah linear.

KERNEL

POLINOMIAL: DARI 2D KE 3D

Contoh kernel polinomial homogen derajat 2 (c=0): fitur kuadrat mengubah pemisahan di ruang asal menjadi linear di ruang fitur.

$x = [x_1, x_2]^T$	dua fitur awal
$x x^T = [[x_1^2, x_1x_2], [x_2x_1, x_2^2]]$	hasil kali titik
$x_1^2 \cdot x_1x_2 \cdot x_2x_1 \cdot x_2^2$	perhatikan suku silangnya
$\varphi(x) = [x_1^2, \sqrt{2} x_1x_2, x_2^2]$	komutatif; pemetaan kernel eksak memberi $\sqrt{2}$ pada suku silang
fitur 2D → fitur 3D	contoh homogen: d = 2, c = 0
data jadi linearly separable – hyperplane bisa dipasang	di ruang baru



Di ruang 3D, dua kelas terpisah rapi — klasifikasi kembali menjadi masalah linear biasa.

PRAKTIK

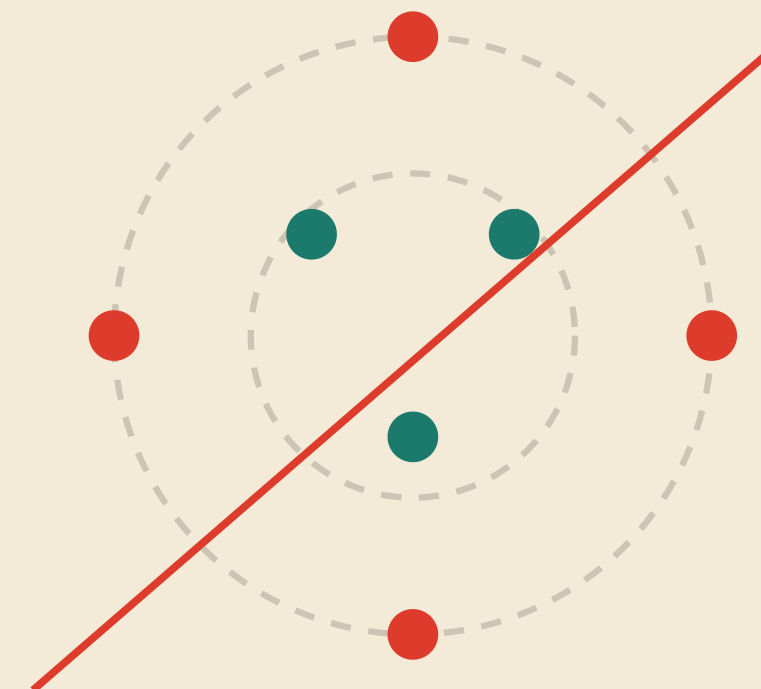
SCIKIT-LEARN: 45% → 100%

Dataset sintetis dua kelas dengan radius 10 dan 5 (400 titik). Split 75/25 → 300 latih, 100 uji.

```
svc = SVC(kernel='linear')
svc.fit(X_train, y_train)
svc.score(X_test, y_test)
→ ± 0,45-0,50
```

```
# fitur kuadrat + refit SVC linear
cols = ['x1', 'x2', 'x1²',
        'x2²', 'x1·x2']
X2 = df[cols]
Xtr, Xte, ytr, yte = train_test_split(
    X2, y, test_size=.25)
svc = SVC(kernel='linear')
svc.fit(Xtr, ytr)
svc.score(Xte, yte) → 1.00
```

KERNEL LINEAR • ± 50%



garis apa pun tetap meleset

SETELAH FITUR KUADRAT • 100%



fitur tambahan = “peta” ke ruang yang lebih tinggi

- Kernel linear pada data lingkaran: garis terbaik pun hanya menebak sekitar setengah.
- Setelah tiga fitur kuadrat ditambahkan, data linearis — akurasi langsung 100% pada data uji.
- SVC = Support Vector Classification; untuk regresi ada SVR.

fitur kuadrat dibuat manual, lalu SVC linear dilatih ulang

REKAP

TIGA METODE, SATU TABEL

Semuanya supervised learning untuk klasifikasi — yang berbeda: cara memandang keputusan dan kapan biayanya dibayar.

ASPEK	KNN	DECISION TREE	SVM
Kategori	Lazy · instance-based	Eager	Eager
Kunci keputusan	Kemiripan: jarak + voting mayoritas	Aturan; split dipilih via entropi & gain	Margin: hyperplane & support vector
Hyperparameter	Nilai k & ukuran jarak	Kedalaman, kriteria split, pruning	Jenis kernel (C, γ di luar cakupan)
Kelebihan	Batas non-linear, tanpa asumsi distribusi	Mudah dibaca sebagai aturan	Margin maksimum membantu generalisasi
Keterbatasan	Sensitif outlier, mahal saat testing	Rawan overfitting, perlu pruning	Perlu pemilihan kernel & QP
Normalisasi	Sering perlu — jarak peka skala	Biasanya tidak perlu	Sering dianjurkan; penting untuk RBF

tiga sudut pandang berbeda untuk masalah yang sama: “data ini kelas apa?”

TUGAS

LATIHAN SEBELUM BERTEMU MINGGU DEPAN

Tiga soal kecil yang memakai konsep dari tiap metode.

KNN

Data latih: A(1,2)+, B(3,1)–, C(0,4)–, D(4,4)+.
Prediksi titik (1,1) dengan k=1 dan k=3 —
apakah hasilnya sama?

Latih: hitung jarak, urutkan, voting.

DECISION TREE

Tambahkan satu sampel noise ke dataset
main tenis. Amati: di node mana pohon harus
tumbuh, dan bagaimana pengaruhnya ke
data uji.

Latih: pahami hubungan noise ↔ overfitting.

SVM

Ekstensi: coba SVC(kernel='poly', degree=2)
pada dataset lingkaran. Bandingkan dengan
demonstrasi fitur kuadrat manual + SVC
linear.

Latih: kernel trick & pemetaan fitur.

TERIMA KASIH

Rangkuman Week 6 — kNN, Decision Tree, SVM • 21 seksi sumber.

WEEK 06
 SELESAI